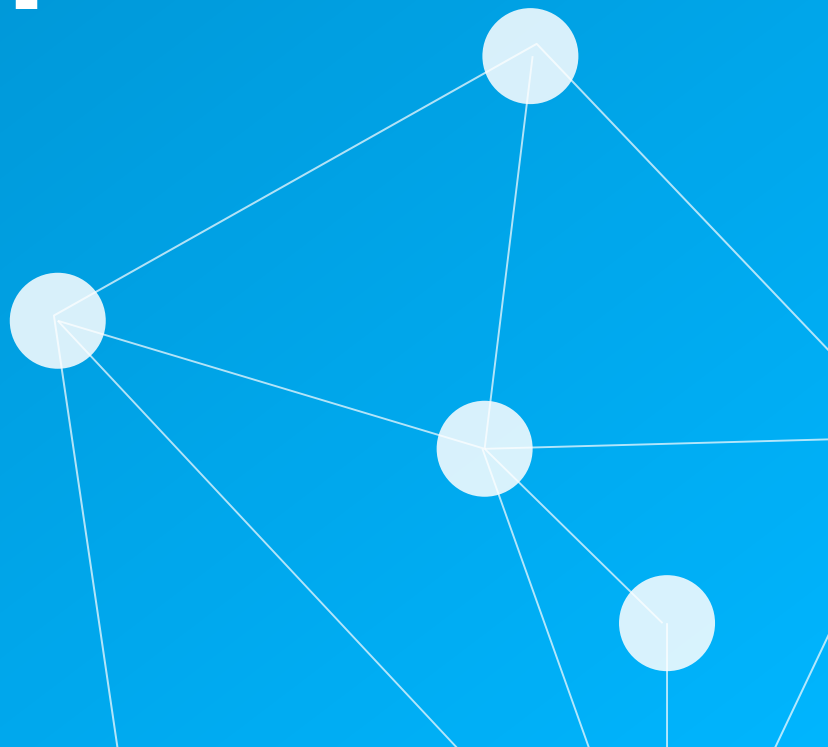
CHAPTER

04

탐색



4장. 탐색



4.1 탐색이란?

4.2 순차 탐색

4.3 이진 탐색

4.4 해싱

4.5 해싱의 오버플로우 해결 방법

4.1 탐색이란?

- 데이터의 집합에서 원하는 조건을 만족하는 데이터를 찾는 작업



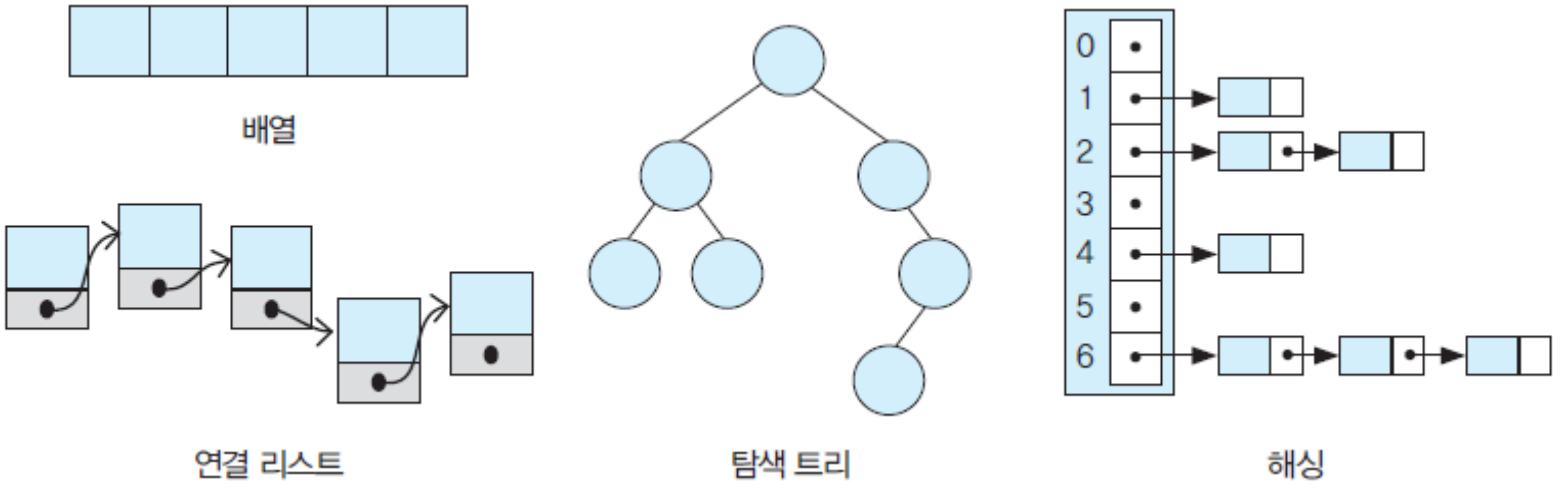




- 탐색
  - 테이블에서 원하는 탐색키를 가진 레코드를 찾는 작업

탐색을 위한 테이블

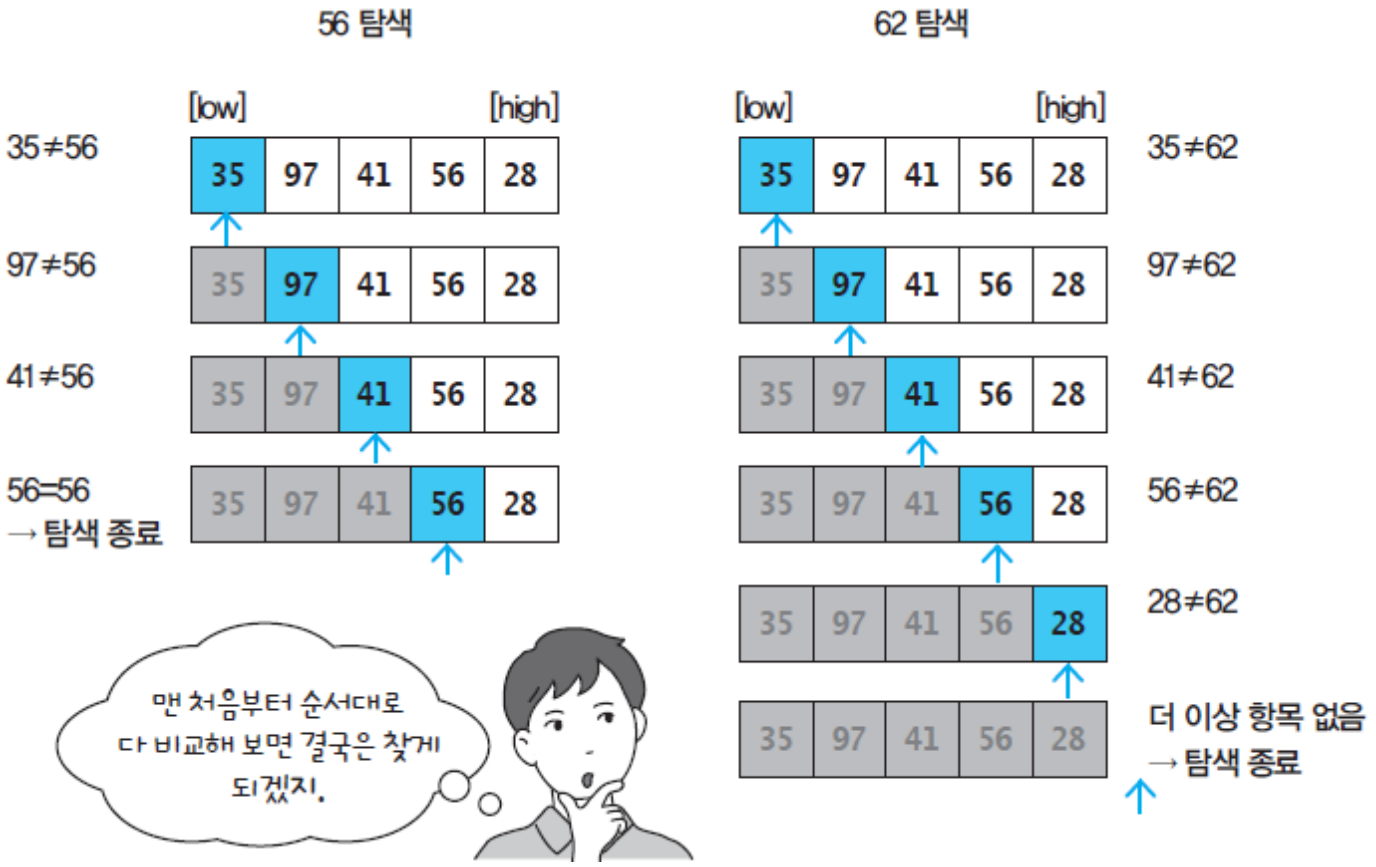
- 테이블은 여러 가지 방법으로 구성할 수 있음
- 테이블을 구성하는 방법에 따라 효율이 달라짐
  - 배열, 연결 리스트, 탐색 트리, 해싱 등



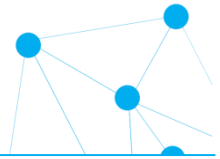
- 탐색 방법 선택 시 고려할 점
  - 탐색 성능, 삽입 성능, 삭제 성능

4.2 순차 탐색

- 순차 탐색(sequential search) 또는 선형 탐색(linear)
  - 일렬로 늘어선 자료(레코드) 중에서 원하는 킷값을 가진 레코드를 순서대로 찾는 알고리즘



순차 탐색 알고리즘



```
01  def sequential_search(A, key):
02      for i in range(len(A)) :
03          if A[i] == key :
04              return i
05      return -1
```

```
# 탐색 성공 : 탐색키의 인덱스 반환
# 탐색 실패  :-1 반환
```

- 복잡도: 입력크기(n), 기본연산(3행) → 입력의 구성에 따라 다름
  - 최선: 첫번째 항목이 key여서 한번의 비교만으로 종료

    $T_{best}(n) = 1 \ \in O(1)$
  - 최악: key가 리스트에 없거나 맨 뒤에 있는 경우

    $T_{worst}(n) = n \ \in O(n)$
  - 평균: 리스트의 모든 숫자가 골고루 key 로 사용되는 경우

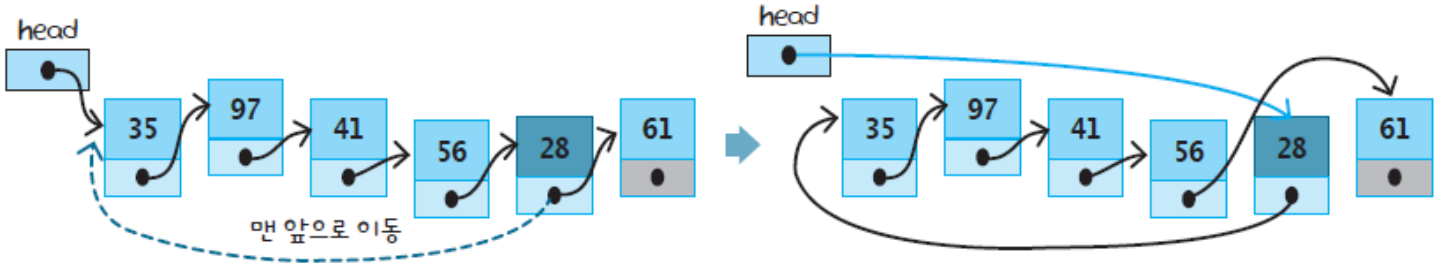
    $T_{avg}(n) = (n+1)/2 \ \in O(n)$

순차 탐색을 개선하는 방법 첫번째

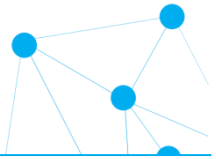
- 자주 사용되는 레코드를 앞쪽으로 옮기는 방법
  - 자기 구성(self-organizing) 순차 탐색
- 맨 앞으로 보내기(move to front)
  - 탐색에 성공한 레코드를 리스트의 맨 앞으로 보내는 방법
  - 배열의 경우



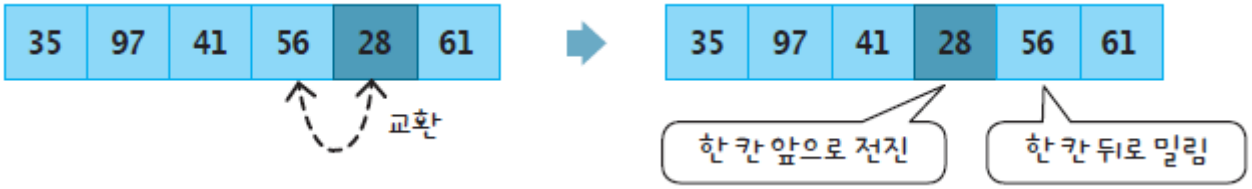
  - 연결 리스트의 경우



순차 탐색을 개선하는 방법 두번째



- 탐색된 레코드는 바로 앞의 레코드와 교환하는 전략
  - 자주 탐색되는 레코드는 앞으로 오게해서 탐색시간 줄

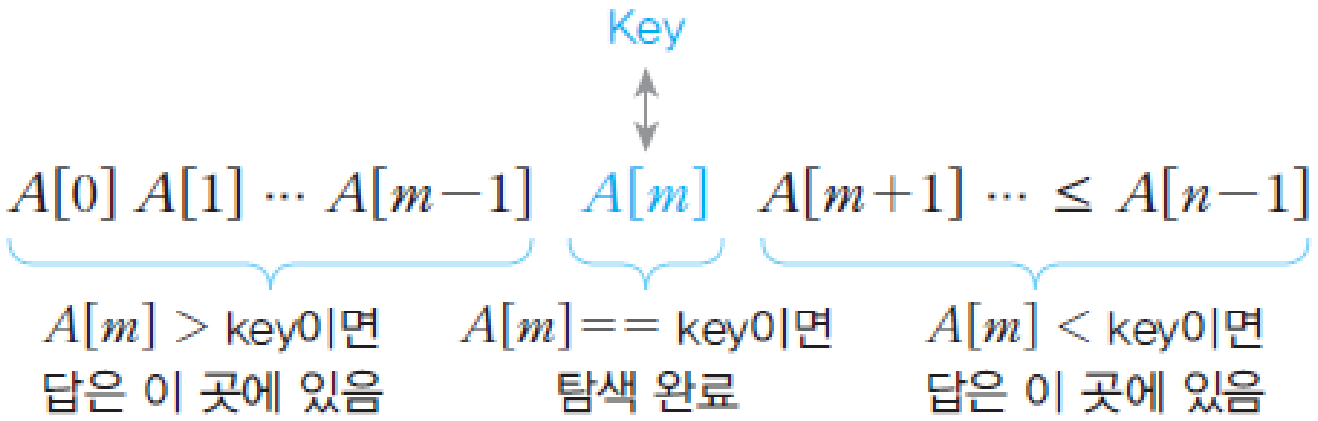


- 알고리즘

```
def sequential_search(A, key) :
    for i in range(len(A)) :
        if A[i] == key :
            if i > 0 :                          # 맨 처음 요소가 아니면
                A[i], A[i-1] = A[i-1], A[i]     # 교환하기
                i -= 1                          # 한칸 앞으로 이동
            return i
    return -1
```

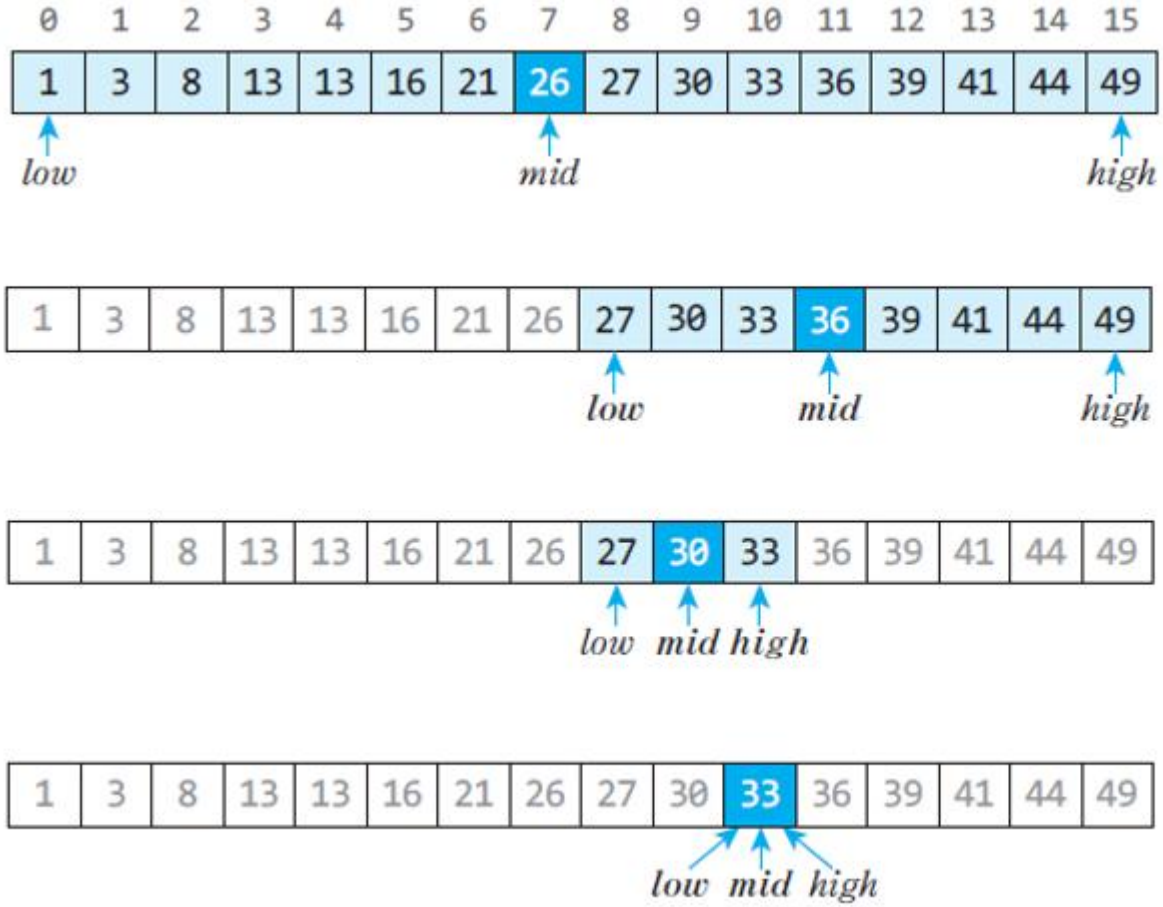
4.3 이진 탐색

- 데이터가 이미 정렬되어 있을때 사용 가능
- 한 번 비교할 때마다 탐색 범위가 절반으로 줄어듦
- 중앙에 있는 항목 먼저 검색 → 답이 있을 부분만 다시 탐색
- 축소 정복 기법



이진 탐색 과정

- Key : 33



최초의 탐색 범위 :
*low=**0**, high=**15**, mid=**7***

❶ A[mid] < 33
low ← *mid*+1

오른쪽 부분 탐색 :
*low=mid+1=**8**, high=**15**, mid=**11***

❷ A[mid] > 33
high ← *mid*−1

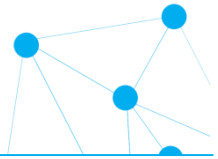
왼쪽 부분 탐색 :
*low=**8**, high=mid-1=**10**, mid=**9***

❸ A[mid] < 33
low ← *mid*+1

오른쪽 부분 탐색 :
*low= mid+1=**10**, high=**10**, mid=**10***

❹ A[mid] = 33
탐색 완료

이진 탐색 알고리즘(순환 구조)

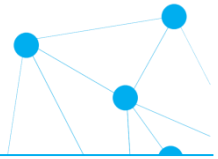


```
01:  def binary_search(A, key, low, high) :
02:      if (low <= high) :              # 항목들이 남아 있으면(종료 조건)
03:          middle = (low + high)//2    # middle 계산
04:          if key == A[middle] :       # 탐색 성공
05:              return middle           # 중앙 레코드의 인덱스 반환
06:          elif (key<A[middle]) :      # 왼쪽 부분리스트 탐색 -> 순환호출
07:              return binary_search(A, key, low, middle - 1)
08:          else :                      # 오른쪽 부분리스트 탐색 -> 순환호출
09:              return binary_search(A, key, middle + 1, high)
10:      return -1                       # 탐색 실패 -1 반환
```

```
listA = [1, 3, 8, 13, 13, 16, 21, 26, 27, 30, 33, 36, 39, 41, 44, 49]
print("입력 리스트 =", listA)    # 리스트는 반드시 정렬되어 있어야 한다.
print("33 탐색(순환) -->", binary_search(listA, 33, 0, len(listA)-1) )
print("32 탐색(순환) -->", binary_search(listA, 32, 0, len(listA)-1) )
```

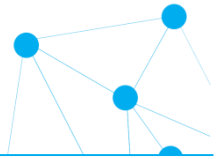
```
C:\WINDOWS\system32\cmd.exe
입력 리스트 = [1, 3, 8, 13, 13, 16, 21, 26, 27, 30, 33, 36, 39, 41, 44, 49]
33 탐색(순환) --> 10
32 탐색(순환) --> -1
```

이진 탐색 알고리즘(반복 구조)



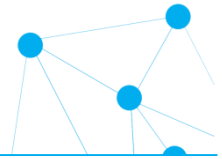
```
01:  def binary_search_iter(A, key, low, high) :
02:      while (low <= high) :          # 항목들이 남아 있으면(종료 조건)
03:          middle = (low + high)//2 # middle 계산
04:          if key == A[middle]:       # 탐색 성공
05:              return middle
06:          elif (key > A[middle]):   # key가 middle의 값보다 크면
07:              low = middle + 1      # middle+1 ~ high 사이 검색
08:          else:                      # key가 middle의 값보다 작으면
09:              high = middle - 1     # low ~ middle-1 사이 검색
10:      return -1                      # 탐색 실패 -1 반환
```

복잡도 분석 및 이진 탐색의 특징



- 탐색 범위 $2^k \rightarrow 2^{k-1} \rightarrow 2^{k-2} \rightarrow \cdots \rightarrow 2^1 \rightarrow 2^0$
- 입력 구성에 따른 차이가 있음
  - 최선 : 바로 찾을 때
    - $T_{best}(n) \in O(1)$
  - 최악: 리스트에 찾는 값이 없는 경우
    - $k = log_2 n$
    - $T_{worst}(n) \in O(log_2 n)$
- 이진 탐색의 특징
  - 반드시 리스트가 정렬되어 있어야 적용 가능함
  - 탐색은 효율적
  - 데이터의 삽입이나 삭제가 빈번한 응용에는 적합하지 않음

연습문제



1. 키값 28을 가지고 아래의 리스트(16개 항목)를 탐색할 때 비교연산횟수를 구하시오.

8	11	12	15	16	19	20	23	25	28	29	31	33	35	38	40

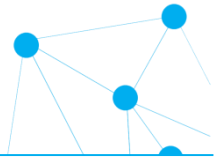
(1) 순차 탐색

(2) 이진 탐색

2. 이진탐색으로 C를 찾을 경우 비교 횟수는? (15개 항목)

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O

4.4 고급 탐색 구조 : 해싱

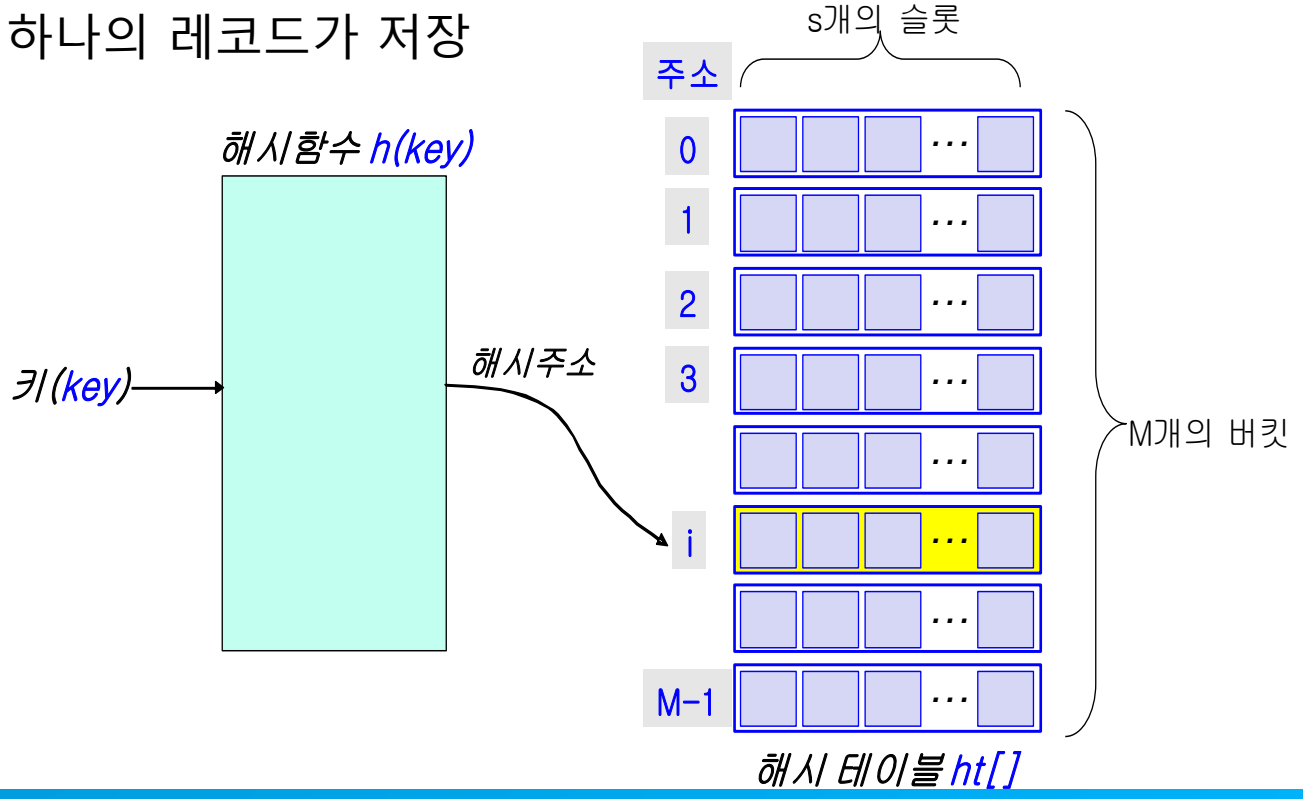


- 해싱(hashing) 이란?
  - 키 값에 대한 산술적 연산에 의해 레코드가 저장될 위치(주소)를 계산 (비교가 아님)
  - 삽입/삭제/탐색 연산은 이 주소에서 이루어짐
    - 아파트의 세대별 우편함과 유사함
  - 이상적인 경우 삽입/삭제/탐색 연산의 시간복잡도 : O(1)

- 실제의 해싱
  - 탐색키가 실수이거나 문자열, 굉장히 큰 정수라면? 메모리 부족!
  - 탐색키를 작은 정수로 대응시키는 함수 필요 → 해시 함수
  - 성능이 O(1)보다 떨어짐

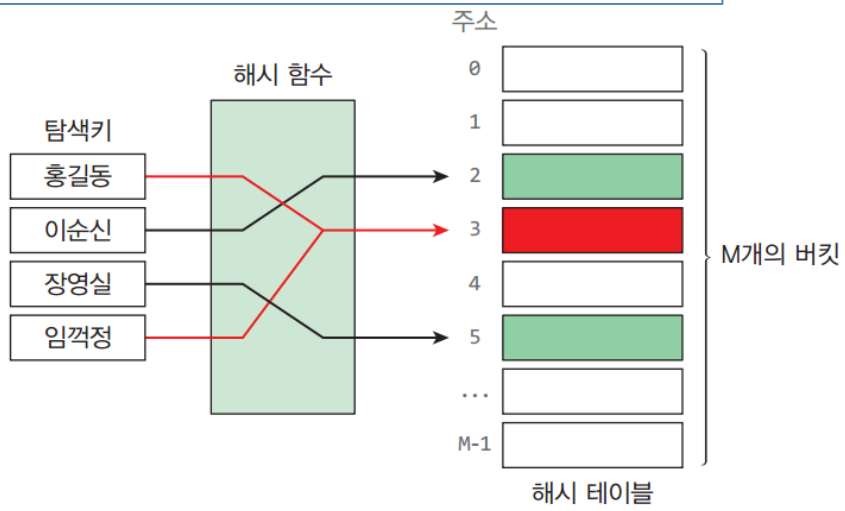
해싱의 구조

- 해싱의 구조
  - 탐색키를 입력 받아 해시 주소(hash address) 생성
  - 해시 함수(hash function) : 레코드가 저장될 위치 계산
  - 해시 테이블 : 레코드를 저장한 테이블로 여러 개의 버킷으로 구성
  - 버킷 : 버킷 당 하나의 주소가 생성, 여러 개의 슬롯으로 구성
  - 슬롯 : 하나의 레코드가 저장



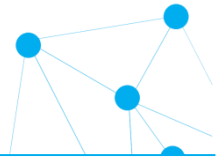
해시 충돌과 오버플로

- 충돌(collsion) : 서로 다른 키가 해시 함수에 의해 같은 주소로 계산
  - 예) h(홍길동)⇒3, h(이순신)⇒2, h(장영실)⇒5, h(임꺽정)⇒3
  - 동의어(synonym)



  - 버킷에 여러 개의 슬롯이 있다면 각각 다른 슬롯에 저장
  - 오버플로(overflow) : 해시 충돌이 버킷의 슬롯 수보다 많은 상황

해시 함수

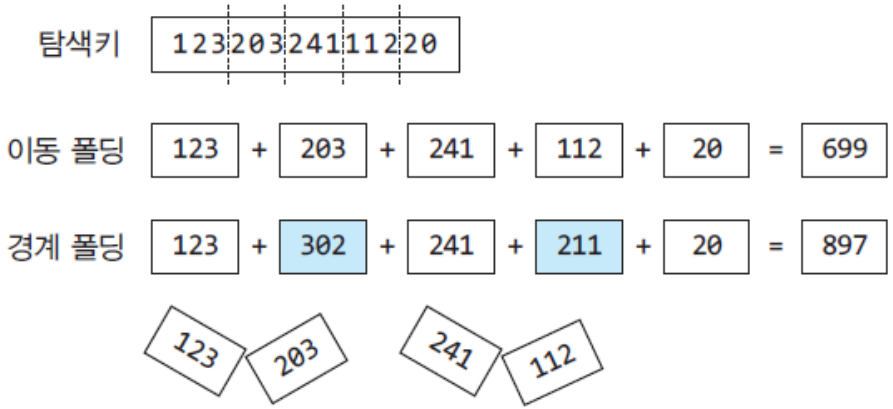


- 좋은 해시 함수의 조건
  - 충돌이 적어야 한다
  - 함수 값이 테이블의 주소 영역 내에서 고르게 분포되어야 한다
  - 계산이 빨라야 한다

- 종류
  - 제산 함수
  - 폴딩 함수
  - 중간 제곱 함수
  - 비트 추출 방법
  - 숫자 분석 방법

해시 함수의 종류

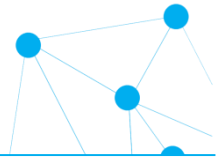
- 제산 함수 : $h[k]=k \bmod M$ (*M : 버킷수*)
  - M은 소수(prime number)가 이상적
- 중간 제곱 함수
  - 탐색키를 제곱한 다음, 중간의 몇 비트를 취해서 해시 주소 생성
- 폴딩 함수 : 해시테이블의 인덱스 크기로 분할 → 오른쪽 끝자리가 일치하도록 옮겨 더함



경계값 뒤집기의 기준은
- 비트를 뒤집거나
- 숫자를 역순으로 배열
- 보수를 취하는 것도 가능

- 비트 추출 함수
  - 탐색키를 이진수로 간주, 임의의 위치의 k개의 비트를 사용
- 숫자 분석 방법
  - 키에서 편중되지 않는 수들을 테이블의 크기에 적합하게 조합

연습문제 : tree



※ 다음 키 중 tree의 버킷 주소를 결정하시오. 버킷수는 256개이며, 알파벳은 아스키코드 값을 사용하시오. **아스키코드 : t(116)+r(114)+ e(101)+ e(101) = 432**

1. **제산함수(M은 256)**

   432 ÷ 256 = 1 …176 tree의 주소 : 176

2. **중간제곱함수(중간 8비트 취함)**

   - 423 = **$(110100111)_2$**

   $(110100111)^2$ = $(1010111010111 10001)_2$

   $(11010111)_2$ = 215 tree의 주소 : 215

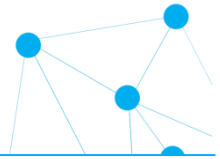
3. **이동폴딩함수**

   - 분할과 합산 : 116 +114 + 101 + 101 = 432
   - 제산함수 적용 : 432 ÷ 256 = 1 …176 tree의 주소 : 176

4. **경계폴딩함수 : 2자리씩 분할하고, 짝수번 수는 뒤집기**

   - 분할 : 11, 61, 14, 10, 11, 01
   - 경계폴딩 적용 : 11 + **16** + 14 + **01** + 11 + **10 = 63**
   - 제산함수 적용 : 63 ÷ 256 = 0 …63 tree의 주소 : 63

연습문제 : best



※ 다음 키 중 tree의 버킷 주소를 결정하시오. 버킷수는 256개이며, 알파벳은 아스키코드 값을 사용하시오. **아스키코드 : b(98) + e(101) + s(115) + t(116)= 430**

1. **제산함수(M은 256)**

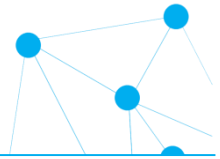
2. **중간제곱함수(중간 8비트 취함)**
   - $430 = (110101110)_2$

     $(110101110)^2 = (10110100100100100100)_2$

3. **이동폴딩함수 : 2자리씩 분할 (전체 숫자를 하나로 이어 붙인뒤 자름)**
   - 분할과 합산 :
   - 제산함수 적용 :

4. **경계폴딩함수 : 3자리씩 분할하고(글자단위로 분할), 짝수번 수는 뒤집기**
   - 분할 :
   - 경계폴딩 적용 :
   - 제산함수 적용 :

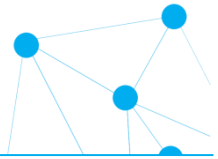
4.5 해싱의 오버플로 해결 방법



- 개방 주소법(open addressing)
  - 오버플로가 일어나면 테이블의 다른 위치에 저장하는 방법
  - 선형 조사법, 이차 조사법, 이중 해싱법 등

- 체이닝(chaining)
  - 하나의 주소에 여러 항목을 저장하도록 테이블의 구조 변경

선형 조사에 의한 오버플로 처리

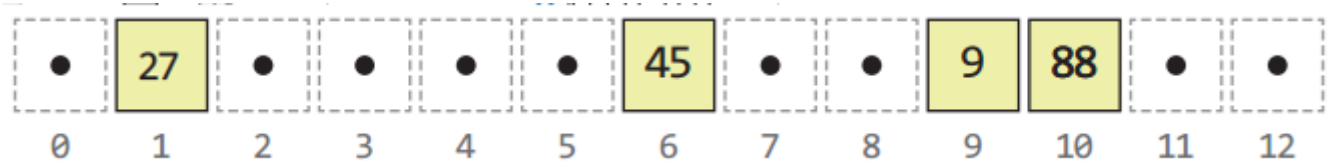


- 충돌이 일어나면 해시 테이블의 다음 위치에 저장
  - 다음 항목을 순서대로 조사: h(k), h(k)+1, h(k)+2,...
  - 빈 곳이 있으면 저장

- 예) 키값이 45, 27, 88, 9, 71, 60, 46, 38, 24인 레코드
  - 테이블의 크기 M=13
  - h(k) = k % M인 경우

key	45	27	88	9	71	60	46	38	24
h(key)	6	1	10	9	6	8	7	12	11

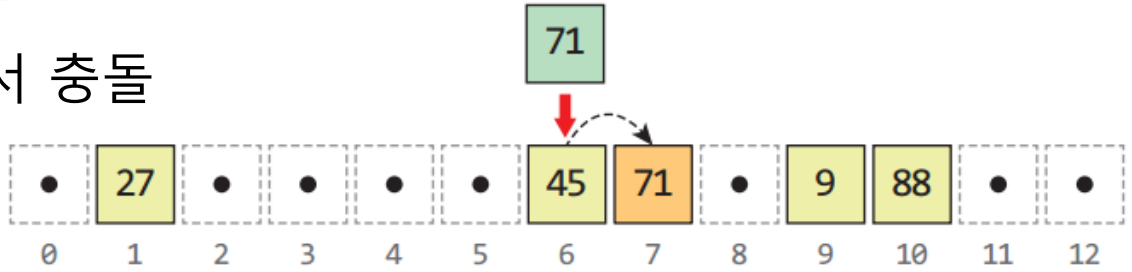
① 45, 27, 88, 9 까지의 삽입



삽입연산

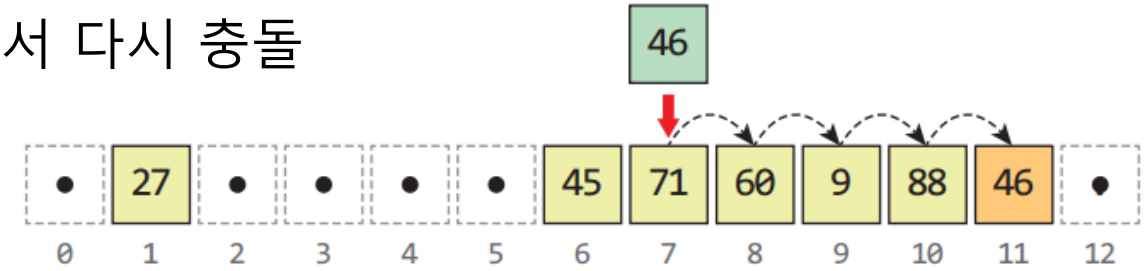
key	45	27	88	9	71	60	46	38	24
h(*key*)	6	1	10	9	6	8	7	12	11

② 71의 삽입에서 충돌

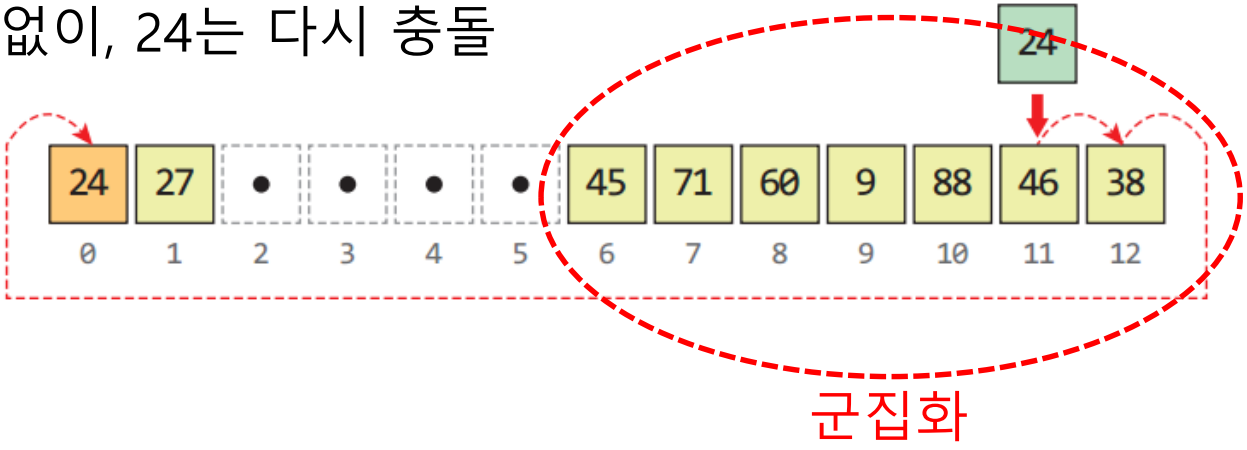


③ 60의 삽입

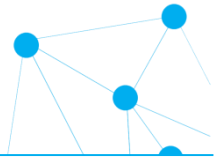
④ 46의 삽입에서 다시 충돌



⑤ 38은 충돌 없이, 24는 다시 충돌



선형 조사 : 삽입 연산 알고리즘



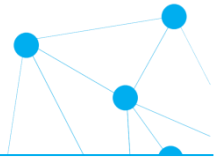
```
M = 13                  # 테이블의 크기
table = [None]*M        # 테이블 만들기: None으로 초기화

def hashFn(key) :       # 해시 함수
    return key % M

def insert(key) : # 삽입 함수
    id = hashFn(key)
    count = M
    while count>0 and table[id] != None : # 비어 있는 버킷이 존재, 다른 키가 삽입
        id = (id + 1 + M) % M             # 다음 위치로 이동
        count -= 1                        # 검사할 남은 위치의 수
    if count > 0 :                        # 비어 있는 버킷 존재하면
        table[id] = key                   # 해당 슬롯에 항목 저장
    return
```

- *삽입 알고리즘 → while count>0 and (table[id] != None and **table[id] != -1**) :*
- *탐색 알고리즘 → if table[id] != -1 and table[id] == key :*

선형 조사 : 삽입 연산 알고리즘



```
print("   최초:", table )
insert(45);  print( "45 삽입:", table )
insert(27);  print( "27 삽입:", table )
insert(88);  print( "88 삽입:", table )
insert(9);   print( " 9 삽입:", table )
insert(71);  print("71 삽입:", table )
insert(60);  print("60 삽입:", table )
insert(46);  print("46 삽입:", table )
insert(38);  print("38 삽입:", table )
insert(24);  print("24 삽입:", table )
```

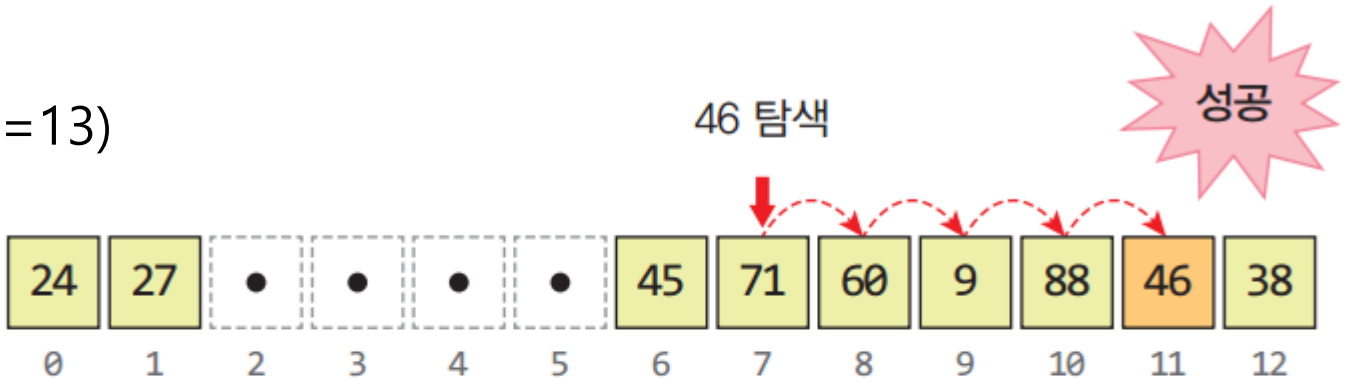
```
   최초: [None, None, None, None, None, None, None, None, None, None, None, None, None]
45 삽입: [None, None, None, None, None, None, 45, None, None, None, None, None, None]
27 삽입: [None, 27, None, None, None, None, 45, None, None, None, None, None, None]
88 삽입: [None, 27, None, None, None, None, 45, None, None, None, 88, None, None]
 9 삽입: [None, 27, None, None, None, None, 45, None, None, 9, 88, None, None]
71 삽입: [None, 27, None, None, None, None, 45, 71, None, 9, 88, None, None]
60 삽입: [None, 27, None, None, None, None, 45, 71, 60, 9, 88, None, None]
46 삽입: [None, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, None]
38 삽입: [None, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
24 삽입: [24, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
```

선형 조사 : 탐색

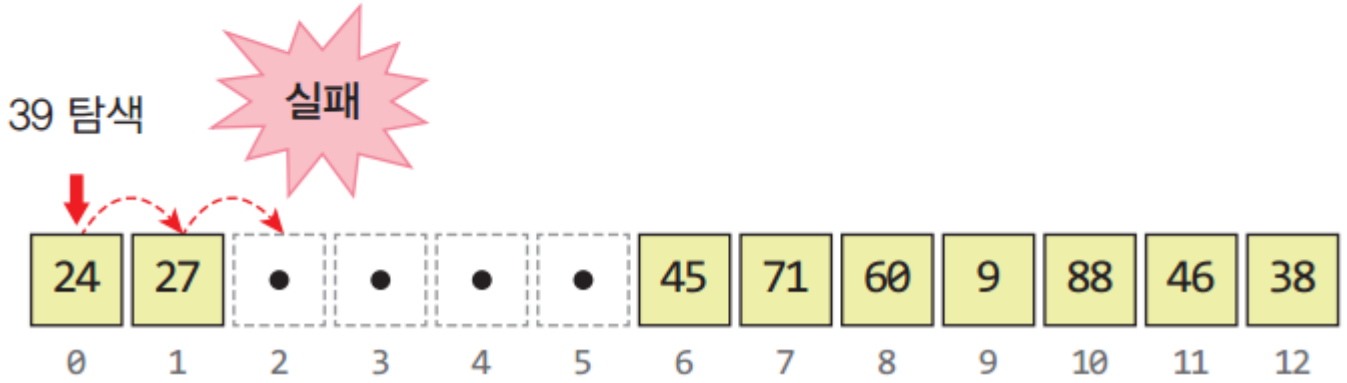
key	45	27	88	9	71	60	46	38	24
h(key)	6	1	10	9	6	8	7	12	11

- 해당키의 레코드를 찾거나, 비어있는 버킷을 만나거나, 모든 버킷을 다 검사할 때까지 진행

- 46 탐색(M=13)



- 39 탐색



선형 조사 : 탐색

key	45	27	88	9	71	60	46	38	24
h(key)	6	1	10	9	6	8	7	12	11

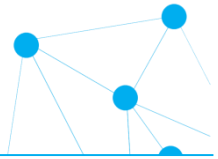
```
M = 13                  # 테이블의 크기
table = [24, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]

def hashFn(key) :   # 해시 함수
    return key % M

def search(key) :
    id = hashFn(key)
    count = M
    while count>0 :
        if table[id] == None :        # 찾는 항목이 테이블에 없음
            return None
        if table[id] == key :         # 찾는 항목이 테이블에 있음 (인덱스 반환)
            return id
        id = (id + 1 + M) % M         # 다음 위치 검사
        count -= 1
    return None
```

- *삽입 알고리즘 → while count>0 and (table[id] != None and tabl*
- *탐색 알고리즘 → if table[id] != -1 and table[id] == key :*

선형 조사 : 탐색 연산 알고리즘

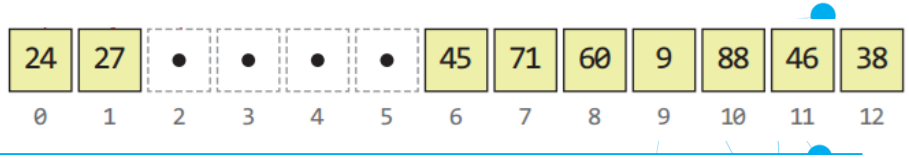


```
print("   최초:", table )
search_data = int(input("탐색할 데이터는? : "))
print(search_data, "탐색:", search(search_data))
```

```
   최초: [24, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
탐색할 데이터는? : 60
60 탐색: 8
```

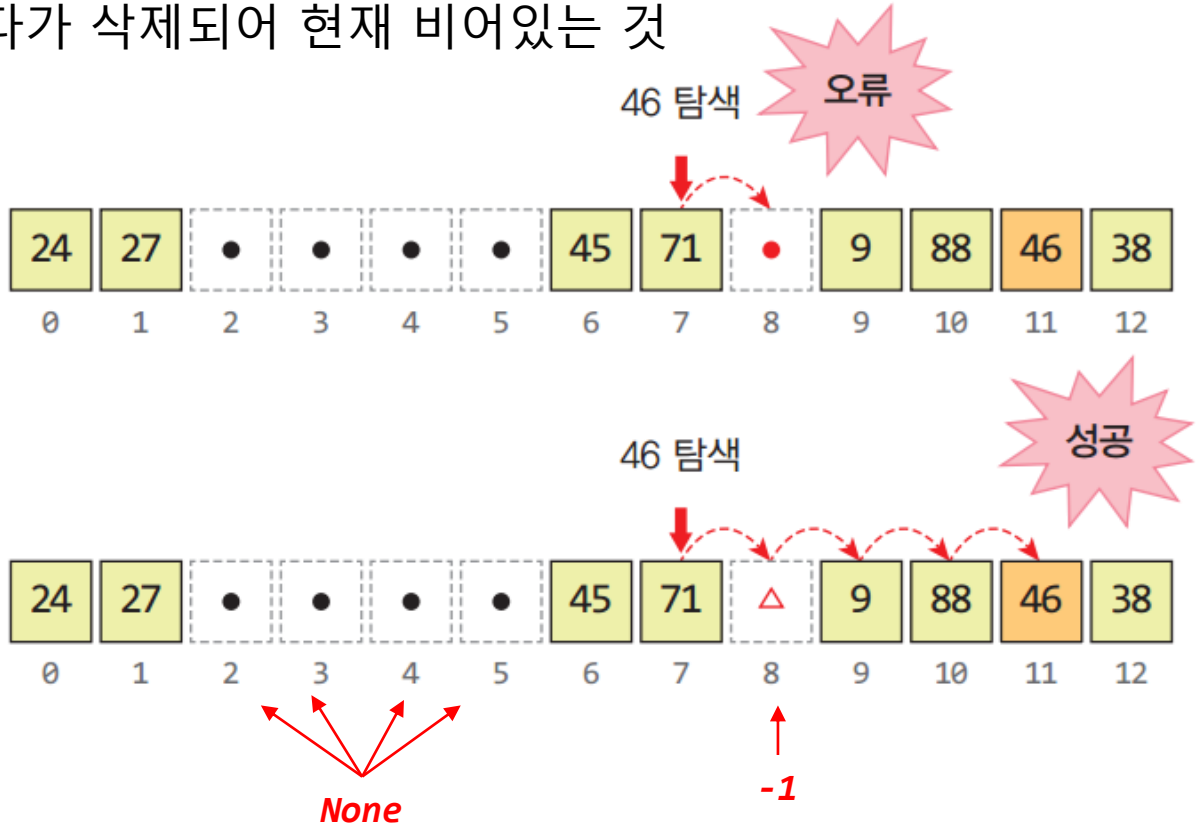
```
   최초: [24, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
탐색할 데이터는? : 47
47 탐색: None
```

선형 조사 : 삭제 연산

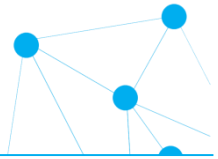


- 60을 먼저 삭제한 상태에서 46을 삭제하려는 경우
- 빈 버킷을 두 가지로 분류해야 함
  - 한 번도 사용하지 않은 것
  - 사용되었다가 삭제되어 현재 비어있는 것

key	45	27	88	9	71	60	46	38	24
h(*key*)	6	1	10	9	6	8	7	12	11



선형 조사 : 삭제 연산 알고리즘



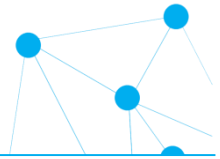
```
M = 13                  # 테이블의 크기
table = [24, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
def hashFn(key) :   # 해시 함수
    return key % M

def delete(key) :
    id = hashFn(key)
    count = M
    while count>0 :
        if table[id] == None : return                  # 키가 없음
        if table[id] != -1 and table[id] == key :  # 키가 있음
            table[id] = -1                             # -1은 사용되었다가 삭제된 버킷
            return                                     # (None 아님)
        id = (id + 1 + M) % M
        count -= 1
```

사용되었다가 삭제된 버킷을 -1로 표시하는 경우 삽입알고리즘과 탐색알고리즘을 수정하시오.

- *삽입 알고리즘 → while count>0 and (table[id] != None and **table[id] != -1**) :*
- *탐색 알고리즘 → if table[id] != -1 and table[id] == key :*

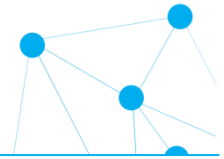
삭제 연산 알고리즘



```
print("   최초:", table )
delete_data = int(input("삭제할 데이터는? : "))
delete(delete_data)
print(delete_data, "삭제:", table)
```

```
   최초: [24, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
삭제할 데이터는? : 60
60 삭제: [24, 27, None, None, None, None, 45, 71, -1, 9, 88, 46, 38]
```

테스트 프로그램



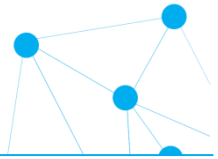
```
data = [45, 27, 88, 9, 71, 60, 46, 38, 26]
for d in data :
    print( "h(%2d)=%2d"%(d,hashFn(d)), end=' ')
    insert(d)
    print(table)

print("46탐색-->", search(46))
print("39탐색-->", search(39))

print("60삭제-->", end='')
delete(60)
print(table)
print("46삭제-->", end='')
delete(46)
print(table)
```

```
C:₩WINDOWS₩system32₩cmd.exe
h(45)= 6 [None, None, None, None, None, None, 45, None, None, None, None, None, None]
h(27)= 1 [None, 27, None, None, None, None, 45, None, None, None, None, None, None]
h(88)=10 [None, 27, None, None, None, None, 45, None, None, None, 88, None, None]
h( 9)= 9 [None, 27, None, None, None, None, 45, None, None, 9, 88, None, None]
h(71)= 6 [None, 27, None, None, None, None, 45, 71, None, 9, 88, None, None]
h(60)= 8 [None, 27, None, None, None, None, 45, 71, 60, 9, 88, None, None]
h(46)= 7 [None, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, None]
h(38)=12 [None, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
h(24)=11 [24, 27, None, None, None, None, 45, 71, 60, 9, 88, 46, 38]
46탐색--> 46
39탐색--> None
60삭제-->[24, 27, None, None, None, None, 45, 71, -1, 9, 88, 46, 38]
46삭제-->[24, 27, None, None, None, None, 45, 71, -1, 9, 88, -1, 38]
```

해싱의 오버플로 해결 방법

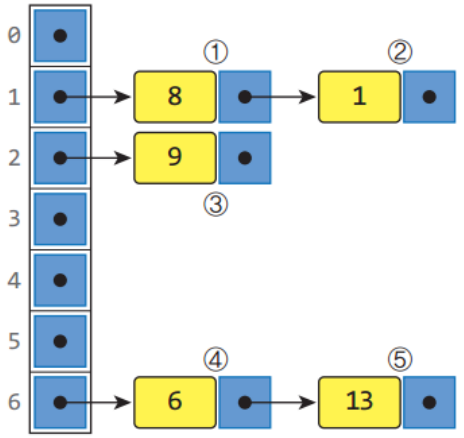


- 개방 주소법(open addressing)
  - 오버플로가 일어나면 테이블의 다른 위치에 저장하는 방법
  - 선형 조사법 : h(k) = k mod M
  - 이차 조사법 : $h'(k) = (h(k) + i^2) \bmod M \quad (i=0, 1, 2, \ldots, M-1)$
    - 군집화 완화
    - 테이블 크기가 소수가 아닌 경우 빈자리를 찾지 못하는 경우 발생
  - 이중 해싱법 : 재해싱 (충돌시 별개의 다른 함수 적용)

- 체이닝(chaining)
  - 하나의 주소에 여러 항목을 저장하도록 테이블의 구조 변경

체이닝(chaining)에 의한 오버플로 처리

- 하나의 버킷에 여러 개의 레코드를 저장
  - 예) h(k)=k%7 을 이용해 8, 1, 9, 6, 13 을 삽입
    - ① 8 저장 : h(8) = 8 % 7 = 1 ⇒ 저장
    - ② 1 저장 : h(1) = 1 % 7 = 1 ⇒ 충돌 ⇒ 새로운 노드 생성 및 저장
    - ③ 9 저장 : h(9) = 9 % 7 = 2 ⇒ 저장
    - ④ 6 저장 : h(6) = 6 % 7 = 6 ⇒ 저장
    - ⑤ 13저장 : h(13) = 13 % 7 = 6 ⇒ 충돌 ⇒ 새로운 노드 생성 및 저장



- 여러 레코드 저장 방법?
  - 연결 리스트나 파이썬의 리스트 등
- 특징
  - 오버플로 발생 → 해당 버킷의 연결 리스트만 처리
  - 필요한 만큼의 메모리만 사용 → 공간적으로 효율적

연습문제

1. 해시 함수는 **h(k) = k mod 11** 이다. **선형조사법**을 이용해 충돌 문제를 해결할 경우 해시 테이블은?

	12	**44**	**13**	**88**	**23**	**94**	**11**	**39**	**20**	**16**	**5**
h(k)	1	0	2	0	1	6	0	6	9	5	5

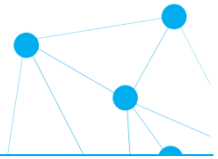
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]

2. 해시 함수는 **h(k) = k mod 7** 이다. 키가 **3, 10, 5, 9, 12**일때 **선형조사법**을 이용해 충돌 문제를 해결할 경우 해시 테이블은? 또한 원래 계산된 슬롯에 저장되지 않는 데이터는?

	3	**10**	**5**	**9**	**12**
h(k)	3	3	5	2	5

	[0]	[1]	[2]	[3]	[4]	[5]	[6]

연습문제 (정답)



1. 해시 함수는 **h(k) = k mod 11** 이다. **선형조사법**을 이용해 충돌 문제를 해결할 경우 해시 테이블은?

	12	**44**	**13**	**88**	**23**	**94**	**11**	**39**	**20**	**16**	**5**
h(k)	1	0	2	0	1	6	0	6	9	5	5

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
	44	**12**	**13**	**88**	**23**	**11**	**94**	**39**	**16**	**20**	**5**

2. 해시 함수는 **h(k) = k mod 7** 이다. 키가 **3, 10, 5, 9, 12**일때 **선형조사법**을 이용해 충돌 문제를 해결할 경우 해시 테이블은? 또한 원래 계산된 슬롯에 저장되지 않는 데이터는?

	3	**10**	**5**	**9**	**12**
h(k)	3	3	5	2	5

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
			9	**3**	**10**	**5**	**12**

10과 12

연습문제

3. 해시 함수는 **h(k) = k mod 11** 이다. **이차조사법**을 이용해 충돌 문제를 해결할 경우 해시 테이블은? $h'(k, i) = h(k) + i^2 \% 11$

	12	44	13	88	23	94	11	39	20	16	5
h(k)	1	0	2	0	1	6	0	6	9	5	5
h'(k, i)											

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]

연습문제(정답)

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
44	12	13	16	88	23	94	39	5	11	20

3. 해시 함수는 **h(k) = k mod 11** 이다. **이차조사법**을 이용해 충돌 문제를 해결할 경우 해시 테이블은? $h'(k, i) = h(k) + i^2 \% 11$

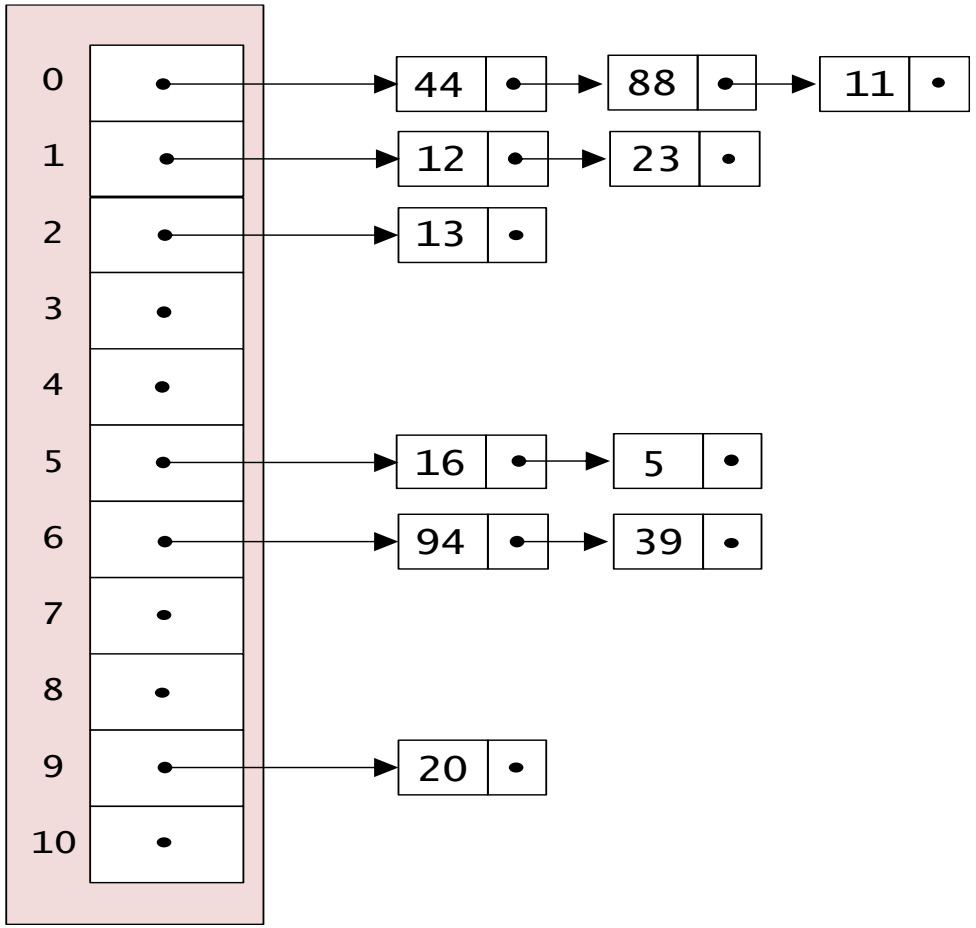
	12	44	13	88	23	94	11	39	20	16	5
h(k)	1	0	2	0	1	6	0	6	9	5	5
h'(k, i)	[1]삽입	[0]삽입	[2]삽입	+1 +4 [4]삽입	+1 +4 [5]삽입	[6]삽입	+1 +4 +9 [9]삽입	+1 [7]삽입	+1 [10]삽입	+1 +4 +9 [3]삽입	+1 +4 +9 +16 +25 [8]삽입

- 88 : 0 → 충돌
  - i=1 : $(0+1^2) \bmod 11 = 1$ → 12 충돌
  - i=2 : $(0+2^2) \bmod 11 = 4$ → [4]에 삽입
- 23 : 1 → 충돌
  - i=1 : $(1+1^2) \bmod 11 = 2$ → 13 충돌
  - i=2 : $(1+2^2) \bmod 11 = 5$ → [5]에 삽입
- 11 : 0 → 충돌
  - i=1 : $(0+1^2) \bmod 11 = 1$ → 12 충돌
  - i=2 : $(0+2^2) \bmod 11 = 4$ → 88 충돌
  - i=3 : $(0+3^2) \bmod 11 = 9$ → [9]에 삽입
- 39 : 6 → 충돌
  - i=1 : $(6+1^2) \bmod 11 = 7$ → [7]에 삽입
- 20 : 9 → 충돌
  - i=1 : $(9+1^2) \bmod 11 = 10$ → [10]에 삽입
- 16 : 5 → 충돌
  - i=1 : $(5+1^2) \bmod 11 = 6$ → 94 충돌
  - i=2 : $(5+2^2) \bmod 11 = 9$ → 11 충돌
  - i=3 : $(5+3^2) \bmod 11 = 3$ → [3]에 삽입
- 5 : 5 → 충돌
  - i=1 : $(5+1^2) \bmod 11 = 6$ → 94 충돌
  - i=2 : $(5+2^2) \bmod 11 = 9$ → 11 충돌
  - i=3 : $(5+3^2) \bmod 11 = 3$ → 16 충돌
  - i=4 : $(5+4^2) \bmod 11 = 10$ → 20 충돌
  - i=5 : $(5+5^2) \bmod 11 = 8$ → [8]에 삽입

연습문제

1. 버킷 크기가 11라고 할때 **체이닝** 방법을 이용해 충돌 문제를 해결시오.

	12	**44**	**13**	**88**	**23**	**94**	**11**	**39**	**20**	**16**	**5**
h(k)	1	0	2	0	1	6	0	6	9	5	5



탐색 방법들의 성능 비교

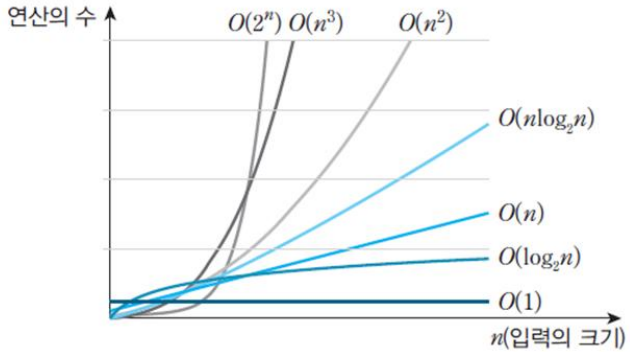
- 해시의 성능 : *O(1)*

$$\alpha = \frac{\text{저장된 항목의 개수}}{\text{해시테이블의 버킷의 수}} = \frac{n}{M}$$

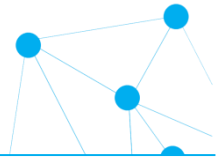
  - 실제 해싱의 탐색 연산은 이상적인 *O(1)* 보다 느림

- 다양한 탐색 방법의 성능 비교

탐색 방법		탐색	삽입	삭제
순차 탐색		$O(n)$	$O(1)$	$O(n)$
이진 탐색		$O(\log_2 n)$	$O(n)$	$O(n)$
이진탐색트리	균형트리	$O(\log_2 n)$	$O(\log_2 n)$	$O(\log_2 n)$
	경사트리	$O(n)$	$O(n)$	$O(n)$
해싱	최선의 경우	$O(1)$	$O(1)$	$O(1)$
	최악의 경우	$O(n)$	$O(n)$	$O(n)$



연습문제



1. 순차 탐색에 대한 설명으로 바른 것은?
   ① 정렬되지 않은 테이블에서도 동작한다.
   ② 최선과 최악의 입력에 대해 시간 복잡도의 차이가 없다.
   ③ 리스트를 균등하게 분할하여 탐색한다.
   ④ 탐색 값과 위치가 비례한다고 가정한다.

2. 다음 중 이진 탐색의 응용 분야로 적절하지 않은 경우는?
   ① 효율적인 탐색이 필요할때 ② 탐색이 매우 빈번하게 발생할 때
   ③ 테이블이 정렬되어 있을때 ④ 데이터의 입력과 삭제가 빈번할 때

3. 해싱에서의 오버플로 처리 방법이다. 나머지와 다른 하나는?
   ① 선형조사법 ② 이차조사법
   ③ 이중해시법 ④ 체이닝

4. 다음 중 좋은 해시 함수의 조건이 아닌 것은?
   ① 코드가 간단해야 ② 충돌이 적어야
   ③ 계산이 빨라야 ④ 주소가 테이블에 고르게 분포되어야



감사합니다!